

Flyfish File Viewer Rich PDF Sample

A compact but realistic multi-page PDF used to verify page navigation, width fitting, dark-mode shell isolation, printing and rendered HTML export.

5

pages for navigation

19

render chains covered

135

extension mappings

0

global style leaks expected

What this PDF should demonstrate

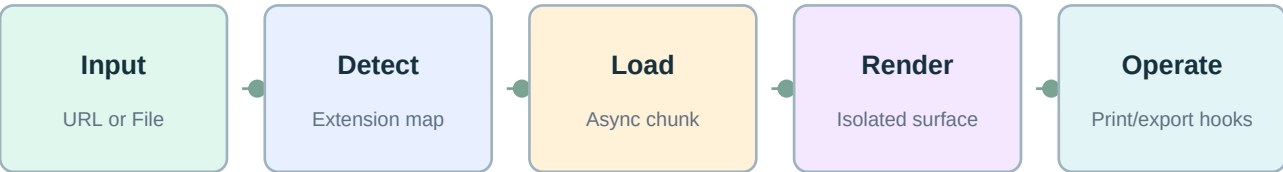
The preview surface should fit to the current viewport width by default. Opening the page sidebar or outline sidebar must not make the document unreadable, and dark mode should not invert the document itself.

Area	How to test	Expected behavior
Toolbar	Change zoom and pages	Buttons stay aligned and readable
Sidebar	Switch page thumbnails and outline	Content keeps usable width
Print	Print rendered HTML	Every page is included
Export	Download rendered HTML	No demo shell is included

1. Rendering Architecture

Flyfish routes each file by extension, then loads only the renderer that is actually needed. This keeps the common path light while still supporting heavy formats such as PDF, OFD, archives, CAD, 3D models and ebooks.

A PDF is already a page-based format, so the viewer uses pdfjs-dist for screen rendering and a dedicated export adapter for complete print output.



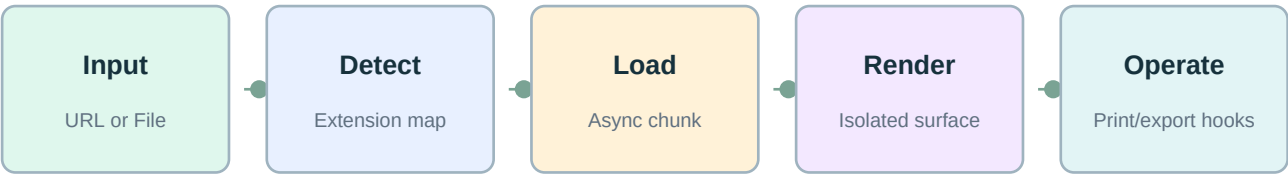
The flow diagram is vector content, so it should stay crisp while zooming in the PDF viewer.

Capability	Implementation	Demo target
PDF	pdfjs-dist	Pages and outline
Word	docx-preview	Responsive paper
Excel	styled-exceljs + e-virt-table	Virtual scrolling
Markdown	marked + GitHub CSS	Theme-aware reading

2. Operation Safety

Business applications often need to check permissions before allowing download, print or export. Operation hooks can return false to cancel the action before any browser side effect happens.

Capability state is dynamic. The host should listen for operation availability instead of assuming that every format can print.



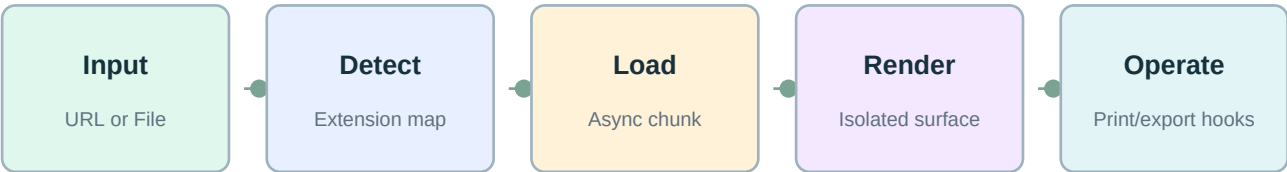
The flow diagram is vector content, so it should stay crisp while zooming in the PDF viewer.

Capability	Implementation	Demo target
download	Original file blob	Available after file load
print	Rendered document	Only complete chains show it
export-html	Rendered HTML package	Available for readable chains
beforeOperation	Permission hook	Can cancel action

3. Theme Boundaries

The demo shell supports light and dark modes. Documents that have their own layout, such as PDF, Word and spreadsheets, keep their original paper or table surface. Markdown and source code adapt because they are viewer-owned reading surfaces.

This separation avoids the classic failure where dark-mode variables make text nearly invisible on a white document page.



The flow diagram is vector content, so it should stay crisp while zooming in the PDF viewer.

Capability	Implementation	Demo target
Demo shell	Theme aware	Panels and buttons adapt
PDF page	Document owned	White page remains white
Markdown	Viewer owned	Light/dark reading theme
Code	Viewer owned	Dark highlighter surface

4. Print Acceptance Notes

Use the toolbar print action to open the rendered print document. The print preview should include this page plus all previous pages, with no current-viewport clipping and no sidebar controls.

Example hook configuration:

```
const options = {
  toolbar: { download: true, print: true, exportHtml: true },
  beforeOperation(context) {
    return context.operation !== 'print' || canPrint(context.filename)
  }
}
```

Excel note: spreadsheets are displayed through a virtual table optimized for large workbooks. Because the DOM does not contain the full sheet at once, the viewer intentionally hides print for Excel until a complete sheet export path is available.

Final check	Expected
Open page sidebar	Document remains readable
Switch dark mode	Demo shell changes, PDF page remains correct
Print from toolbar	All pages are in the print document
Download original	The original PDF is saved